

Nakamoto Proof of Work

Author: Shaheer Ahmad

The Core Problem PoW Solves

Framing: In a decentralised network with no authority, how do thousands of anonymous nodes agree on a single version of truth — without trusting each other?

This is the **Byzantine Generals Problem** applied to open networks. Nakamoto's insight was to replace *trust* with *cost*. Instead of asking “do I trust this node?”, every node asks:

“Did this node spend real-world resources to produce this block?”

Lying becomes expensive. Honest participation becomes profitable. This is the economic engine of Bitcoin.

The Hash Function — Foundation of Everything

What It Does

A cryptographic hash function H maps an input of *any* size to a *fixed-size* output:

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$$

Bitcoin uses **SHA-256**, producing a 256-bit output.

The Three Critical Properties

1. **Deterministic:** Same input always produces same output.
2. **Avalanche Effect:** Changing one bit of input changes $\approx 50\%$ of output bits. No pattern, no gradient to follow.
3. **One-Way (Preimage Resistance):** Given output y , finding input x such that $H(x) = y$ is computationally infeasible.

Key Insight: Because hashes are one-way and unpredictable, the *only* strategy to find a valid nonce is brute-force guessing. There is no smarter path. This is not a weakness — it is the *design*.

The Asymmetry That Powers Bitcoin

Action	Who	Cost
Find a valid nonce	Miner	$\mathcal{O}(2^{256}/T)$ hash attempts
Verify a valid nonce	Any node	$\mathcal{O}(1)$ — one hash computation

This is the **fundamental asymmetry**: producing proof is hard; checking proof is trivial.

The Block Header — What Gets Hashed

A miner does not hash only the nonce. They hash the entire **block header**:

$$\text{block_hash} = H(\underbrace{\text{prev_hash}}_{\text{chain link}} \parallel \underbrace{\text{merkle_root}}_{\text{all transactions}} \parallel \underbrace{\text{timestamp}}_{\text{time context}} \parallel \underbrace{\text{target}}_{\text{difficulty}} \parallel \underbrace{\text{nonce}}_{\text{variable guess}})$$

Why Each Field Matters

- **prev_hash**: Chains blocks together. Altering any past block changes its hash, which breaks every subsequent block's **prev_hash**. Tamper-evidence by design.
- **merkle_root**: A single 256-bit fingerprint representing *all* transactions in the block, computed via a Merkle Tree. Changing any transaction changes the root, which changes the block hash, which invalidates the proof of work.
- **nonce**: A 32-bit integer. The only field the miner freely varies. All other fields are fixed for a given block candidate.

Common Misconception (corrected in session): The nonce is not a secret. It is broadcast publicly with every valid block. The nonce does not need to be random — it just needs to be incremented/varied. Nonce reuse does not create a cryptographic vulnerability here (contrast with ECDSA signing nonces where reuse leaks the private key algebraically).

The Difficulty Target — The Puzzle Specification

Formal Definition

A block is **valid** if and only if:

$$H(\text{block_header}) < T$$

where T is the **target** — a 256-bit threshold value.

Intuition: Think of T as the “winning zone.” Hashes are uniformly distributed across $[0, 2^{256})$. The smaller T is, the tinier the winning zone, and the more guesses required on average.

Leading Zeros Interpretation

Requiring k leading zeros in the hash is equivalent to:

$$T = \frac{2^{256}}{16^k}$$

More leading zeros $\Rightarrow$ smaller $T \Rightarrow$ harder puzzle.

Difficulty as a Ratio

Difficulty D is defined relative to the genesis block target T_0 :

$$D = \frac{T_0}{T_{\text{current}}}$$

Higher D means smaller T means harder puzzle. Genesis block has $D = 1$ by definition.

Expected Work Per Block

Since hash outputs are uniformly distributed, the expected number of hashes to find a valid nonce is:

$$\mathbb{E}[\text{hashes per block}] = \frac{2^{256}}{T} = D \cdot 2^{32}$$

The Difficulty Adjustment Algorithm

The Problem It Solves

Bitcoin targets a mean inter-block time of $\tau = 600$ seconds (10 minutes). As the global hashrate H_{net} (hashes/second) changes, the actual block time drifts:

$$\mathbb{E}[\text{block time}] = \frac{2^{256}}{T \cdot H_{\text{net}}}$$

If H_{net} doubles, block time halves. The network must compensate.

Key Insight — Inferring Hashrate

Nodes cannot directly observe H_{net} . They infer it from *block timestamps*. If blocks arrive faster than expected, H_{net} has increased. If slower, H_{net} has decreased.

$$\hat{H}_{\text{net}} \propto \frac{1}{\text{mean inter-block time}}$$

The 2016-Block Epoch

Bitcoin recalibrates every $E = 2016$ blocks:

$$\text{Epoch duration} = 2016 \times 10 \text{ min} = 20,160 \text{ min} = 336 \text{ hours} = \mathbf{14} \text{ days}$$

14 days is intentional: long enough to smooth variance, short enough to track real hashrate trends.

The Adjustment Formula

At the end of each epoch, every node independently computes:

$$T_{\text{new}} = T_{\text{old}} \times \frac{t_{\text{actual}}}{t_{\text{target}}}$$

where:

- t_{actual} = actual time taken to mine the last 2016 blocks (in seconds)
- $t_{\text{target}} = 2016 \times 600 = 1,209,600$ seconds

Equivalently, in terms of difficulty:

$$D_{\text{new}} = D_{\text{old}} \times \frac{t_{\text{target}}}{t_{\text{actual}}}$$

Bitcoin enforces bounds: $T_{\text{new}} \in \left[\frac{T_{\text{old}}}{4}, 4 \cdot T_{\text{old}} \right]$ to prevent extreme oscillation.

Chain Selection — The Heaviest Chain Rule

The Fork Problem

When two miners find valid blocks simultaneously, the network temporarily holds two competing valid chains. Every node must independently resolve which chain to follow — with no coordinator.

The Rule

Theorem 1 (Nakamoto Chain Selection). *An honest node always extends the chain with the greatest **cumulative proof of work** (chainwork), defined as:*

$$\text{chainwork}(\mathcal{C}) = \sum_{i=0}^{|\mathcal{C}|} W_i \quad \text{where} \quad W_i = \frac{2^{256}}{T_i}$$

where T_i is the target of block i and W_i is the expected number of hashes required to produce it.

Common Misconception: This is often called the “longest chain rule” — but that is imprecise. It is the **heaviest chain rule**. A shorter chain of high-difficulty blocks can outweigh a longer chain of low-difficulty blocks.

3 hard blocks $\succ$ 10 easy blocks

Why This Defeats Attackers

To rewrite history (a 51% attack), an attacker must:

1. Secretly re-mine all blocks from the target block onward.
2. Accumulate more chainwork than the honest network in that same period.

This requires the attacker to control $> 50\%$ of H_{net} — an economic cost proportional to the total mining infrastructure of the entire network. At scale, this is prohibitively expensive.

The Complete PoW Protocol — Summary

The Nakamoto PoW Protocol in Five Lines:

1. Collect pending transactions, compute Merkle root.
2. Assemble block header including `prev_hash`, Merkle root, timestamp, target, and nonce.
3. Repeatedly hash with varying nonce until $H(\text{header}) < T$.
4. Broadcast the valid block. Every node verifies in $\mathcal{O}(1)$.
5. Every 2016 blocks, all nodes independently recompute T and converge on the same value.
6. At any fork, all nodes select the chain with greatest $\sum W_i$.

Project 2 Connections — Testbed Design Notes

These are the **measurable quantities** your SimPy testbed must capture for PoW:

Quantity	Symbol	What It Measures
Network hashrate	H_{net}	Total hashes/sec across all miners
Mean block time	$\bar{t}$	Health of difficulty calibration
Block time variance	σ_t^2	Stability of the protocol
Stale block rate	λ_{stale}	Network propagation quality
Chainwork at time t	$\mathcal{W}(t)$	Security accumulation over time
Fork frequency	f_{fork}	Consensus convergence rate
Difficulty epoch drift	ΔD	Adjustment responsiveness